Time Analysis



Thareendhra Keerthi Wijayasiriwardhane

Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk





Learning Outcomes

- On completion of this session, you should be able to
  - Describe the best case, average case and worst case running times of an algorithm
  - Explain linear, logarithmic, linear logarithmic, quadratic, polynomial, exponential and factorial functions of time complexity
  - Express the running time of an algorithm in Asymptotic notation
  - Express the worst case running time of an algorithm in Big Oh notation
  - Analyze classical searching and sorting algorithms

Running Times

- Best case running time
  - Time taken when the best possible thing happens
  - T(n) would be minimum over instance characteristic n
- Worst case running time
  - Time taken when the worst possible thing happens
  - T(n) would be maximum over instance characteristic n
- Average case running time
  - Time taken for some probability distribution of instance characteristic n
  - T(n) would be the average over instance characteristic n

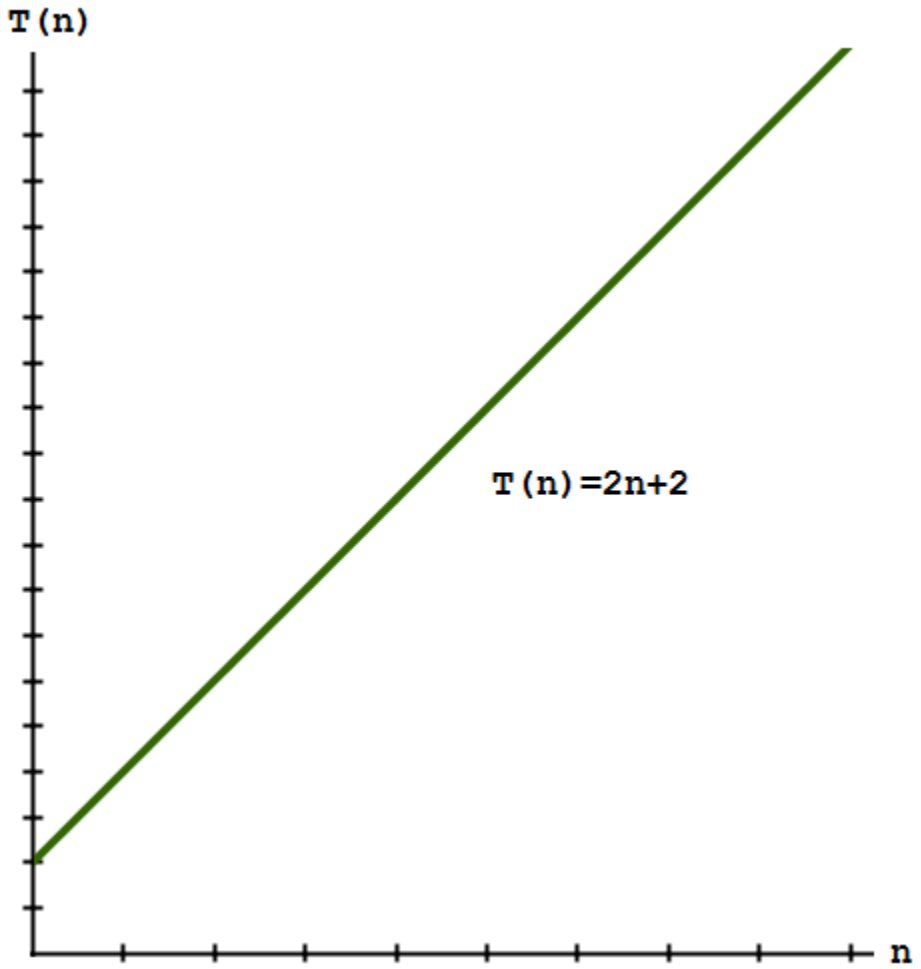
Functions of Time Complexity

- Time complexity of an algorithm is a function of n
- Since n denotes instance characteristic, always $n \geq 0$
- Since time is nonnegative, $T(n) \geq 0$ for all n
- Why are we interested?
  - Would like to know whether the algorithm becomes reasonably or incredibly slow when the problem grows

Linear Function

- Example 1

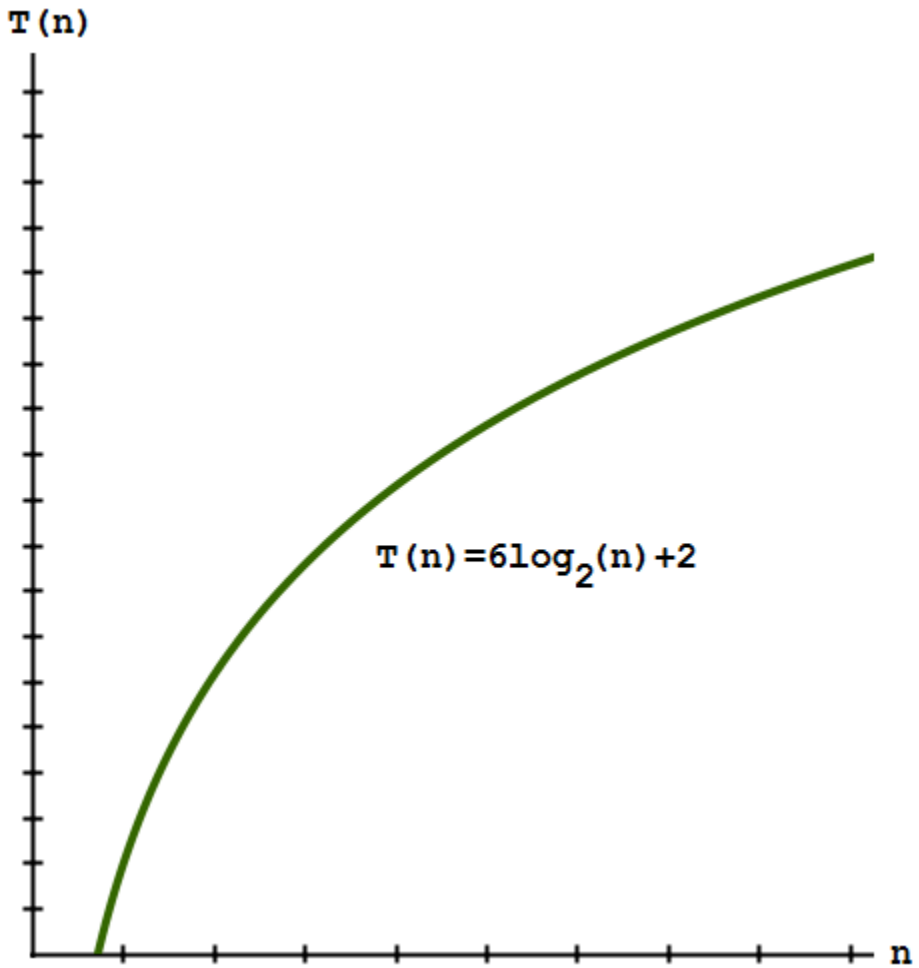
```
1  i ← 1
2  LOOP (i <= n)
3      i ← i + 1
4  END LOOP
```



Logarithmic Function

- Example 2

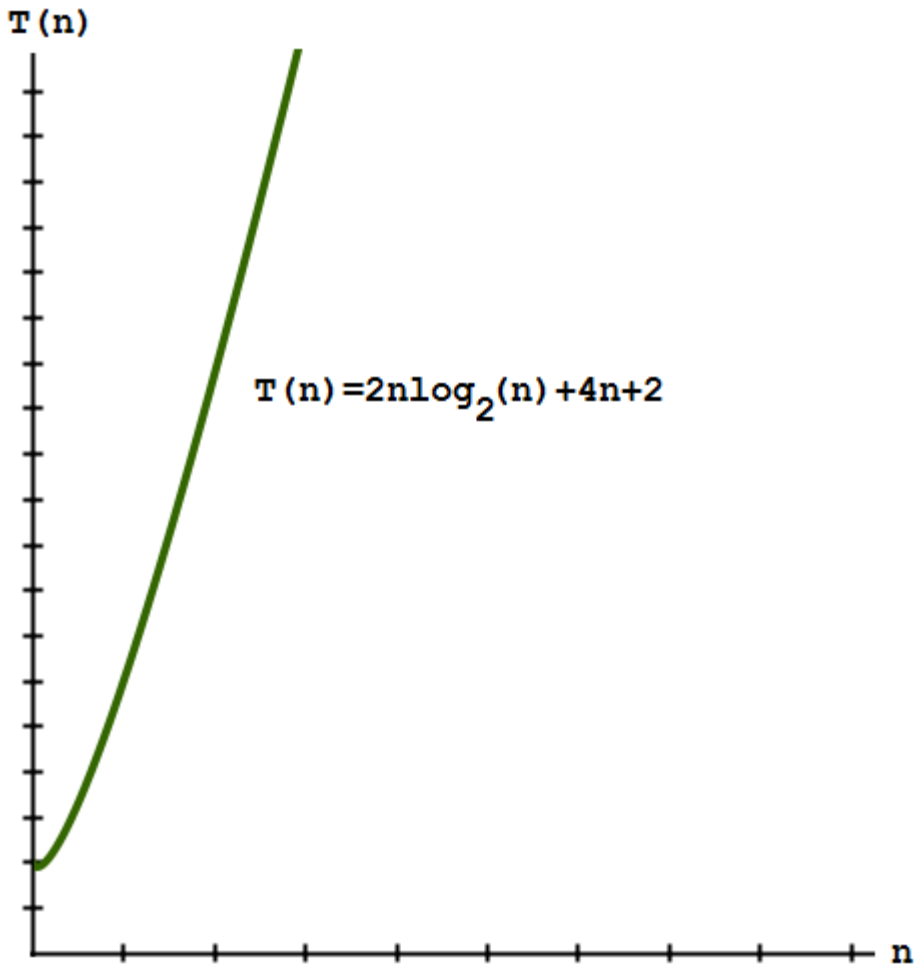
```
1  i ← 1
2  LOOP (i <= n)
3      instruction 1
4      instruction 2
5      instruction 3
6      instruction 4
7      i ← i * 2
8  END LOOP
```



Linear Logarithmic Function

- Example 3

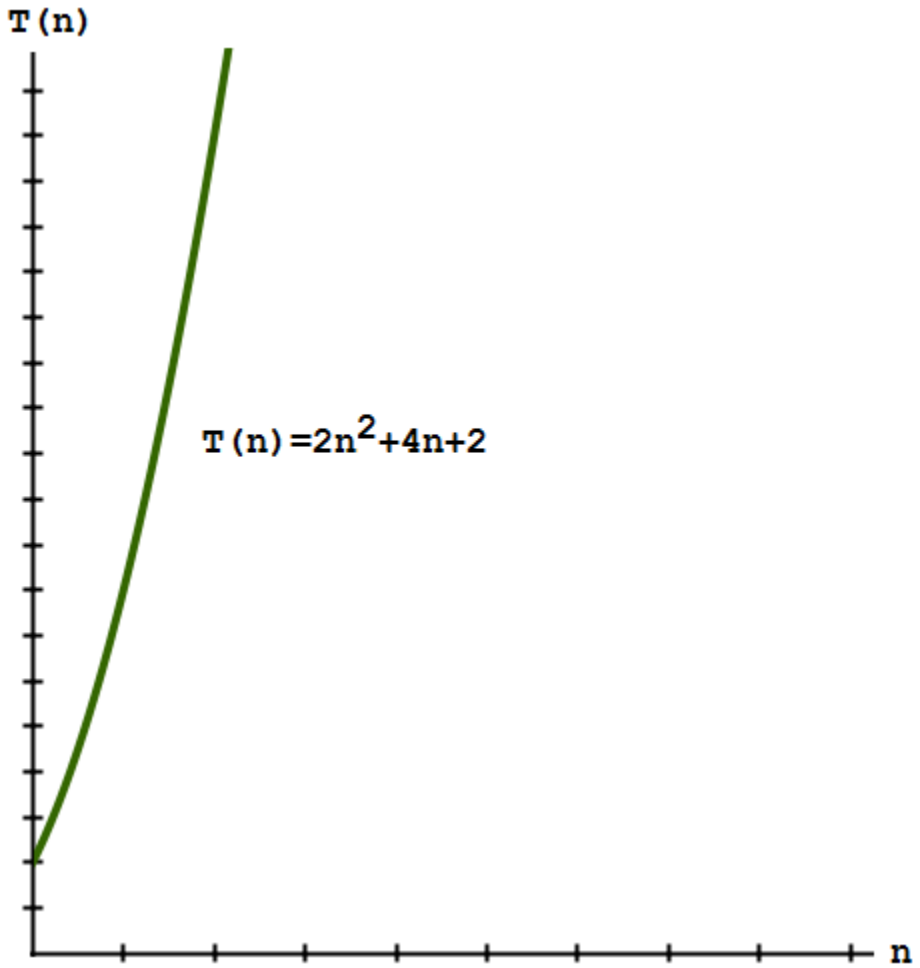
```
1  i ← 1
2  LOOP (i <= n)
3      j ← 1
4      LOOP (j <= n)
5          j ← j * 2
6      END LOOP
7      i ← i + 1
8  END LOOP
```



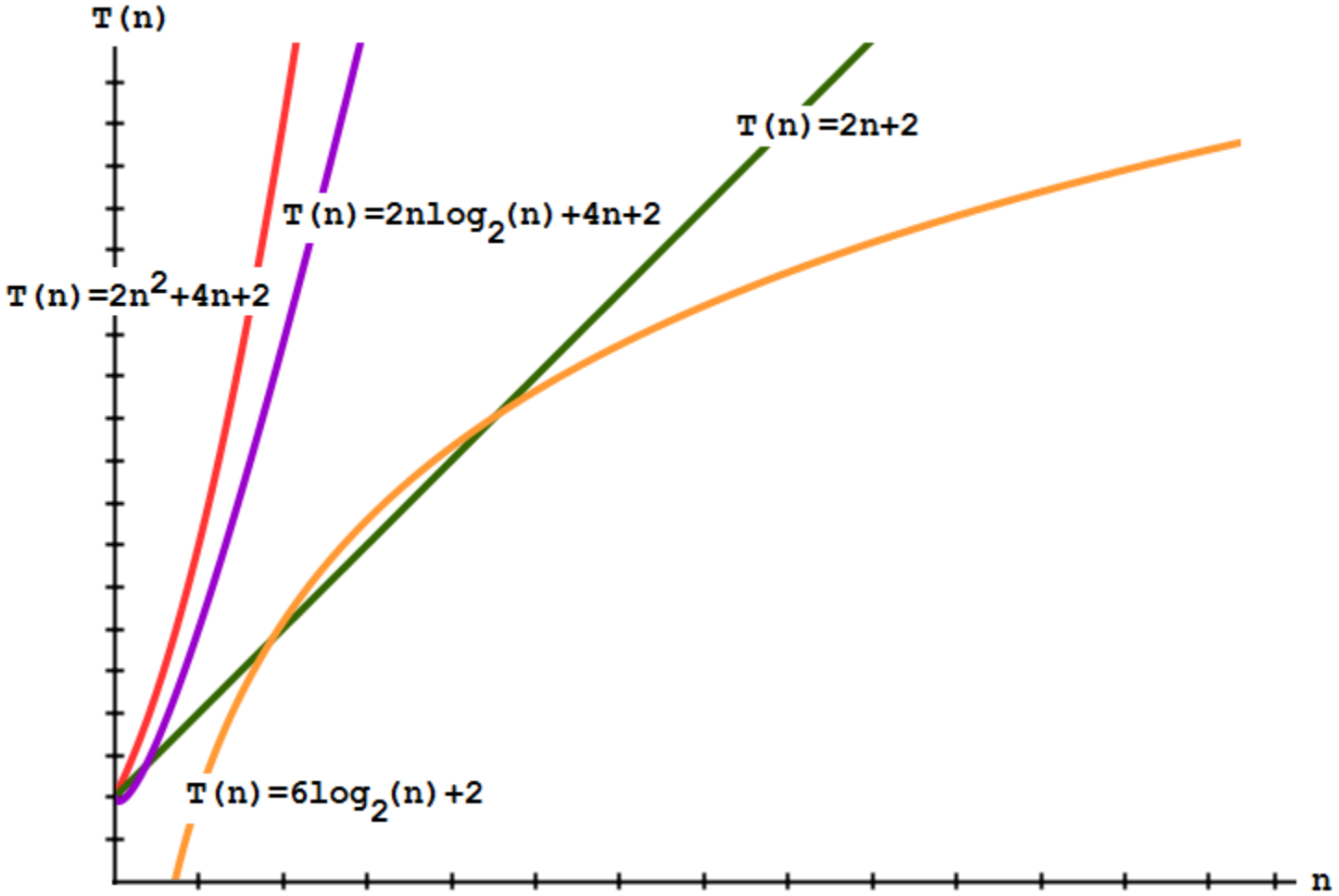
Quadratic Function

- Example 4

```
1  i ← 1
2  LOOP (i <= n)
3      j ← 1
4      LOOP (j <= n)
5          j ← j + 1
6      END LOOP
7      i ← i + 1
8  END LOOP
```

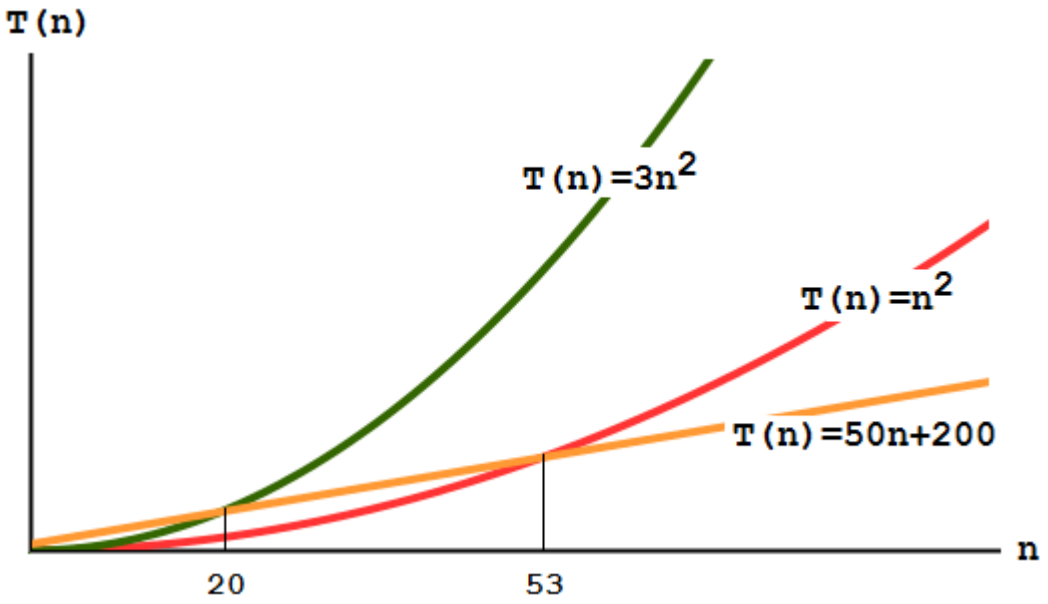


Time Analysis



Asymptotic Notation

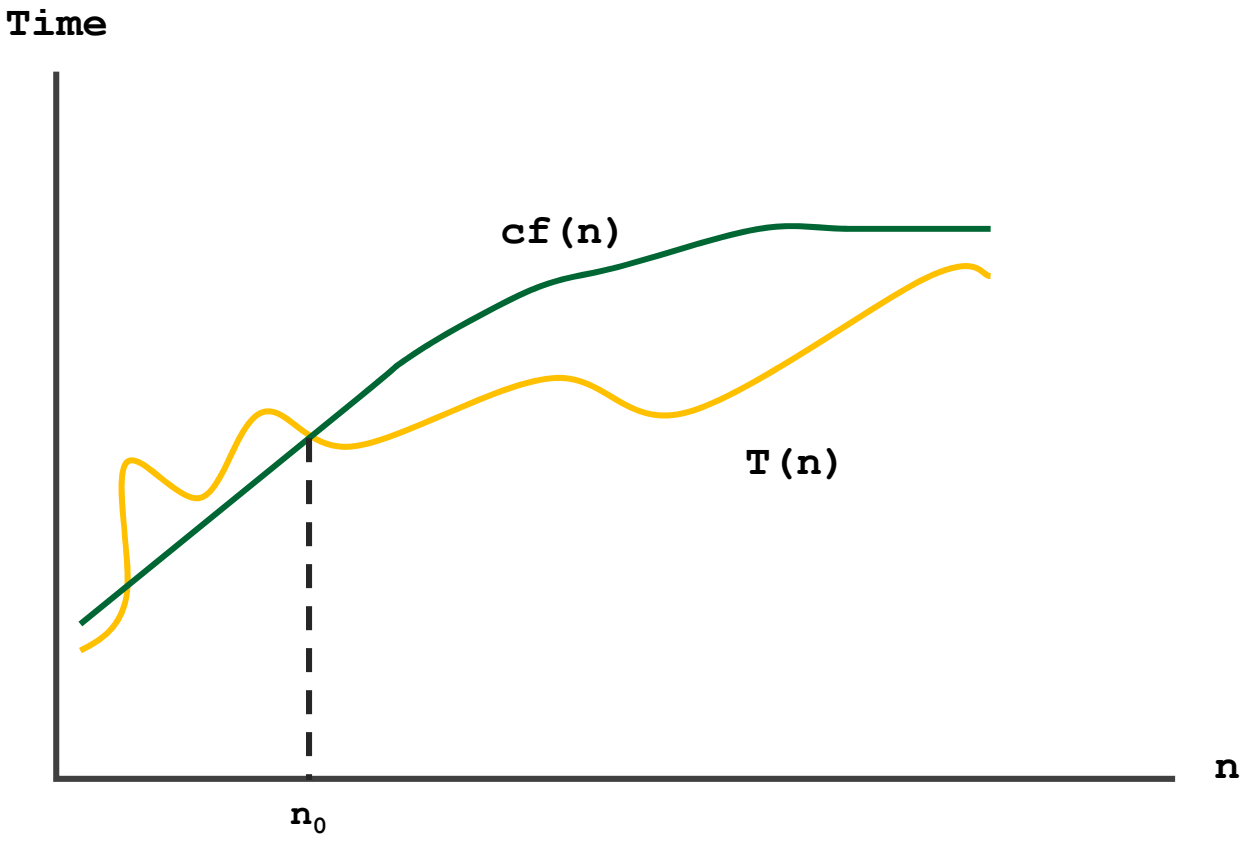
- Expresses an algorithm's running time as its growth rate by dropping the less significant terms and constant coefficients
- Growth rate
  How fast this function grows with the input size
- $\lim_{n\to\infty}\left(\frac{50n+200}{3n^2}\right) = 0$



Big Oh Notation

- Asymptotic notation for the worst case running time of an algorithm
- Definition
  For $T(n) \geq 0$ and $f(n) \geq 0$, $T(n)$ is said to be $O(f(n))$ (big Oh of f of n) if there exist positive constant c and n_0, such that $T(n) \leq cf(n)$ for all $n \geq n_0$

Big Oh Notation



Big Oh Notation

- Example 1
  - $T(n) = 3n + 2$
  - $T(n) = 3n + 2 \leq 3n + n = 4n$ for $n \geq 2$
  - $T(n) \leq 4n = cf(n)$ for $c = 4$ and $n \geq 2$
  - $T(n) = O(f(n)) = O(n)$
- Example 2
  - $T(n) = 10n^2 + 4n + 2$
  - $T(n) = 10n^2 + 4n + 2 \leq 10n^2 + 5n$ for $n \geq 2$
  - $T(n) \leq 10n^2 + 5n \leq 10n^2 + n^2 = 11n^2$ for $n \geq 5$
  - $T(n) \leq 11n^2 = cf(n)$ for $c = 11$ and $n \geq 5$
  - $T(n) = O(f(n)) = O(n^2)$

Big Oh Notation

- Example 3
  - $T(n) = 4n^3 + 2n^2$
  - $T(n) = 4n^3 + 2n^2 \leq 4n^3 + n^3 = 5n^3$ for $n \geq 2$
  - $T(n) \leq 5n^3 = cf(n)$ for $c = 5$ and $n \geq 2$
  - $T(n) = O(f(n)) = O(n^3)$
- Example 4
  - $T(n) = ½ (n + 1)$
  - $T(n) = ½ (n + 1) \leq ½ (n + n) = n$ for $n \geq 1$
  - $T(n) \leq n = cf(n)$ for $c = 1$ and $n \geq 1$
  - $T(n) = O(f(n)) = O(n)$

Searching Algorithms

- Sequential Search

```
ALGORITHM Search(REF list[] <INTEGER>, size <INTEGER>,
        target <INTEGER>)
   Searches target in unordered list and returns location
   if found, else -1
1  location ← -1
2  i ← 0
3  LOOP (i < size AND target ≠ list[i])
4     i ← i + 1
5  END LOOP
6  IF (i ≠ size)
7     location ← i
8  END IF
9  RETURN location
END Search
```

Searching Algorithms

- Binary Search

```
ALGORITHM Search(REF list[] <INTEGER>, size <INTEGER>, target <INTEGER>)
Searches target in ordered list and returns location if found, else -1
1   first ← 0
2   last ← size - 1
3   LOOP (first ≤ last)
4      mid ← (first + last) / 2
5      IF (target > list[mid])
6         first ← mid + 1
7      ELSE IF (target < list[mid])
8         last ← mid - 1
9      ELSE
10        first ← last + 1
11     END IF
12  END LOOP
13  IF (target ≠ list[mid])
14     mid ← -1
15  END IF
16  RETURN mid
END Search
```

Sorting Algorithms

- Selection Sort

```
ALGORITHM Sort(REF list[] <INTEGER>, size <INTEGER>)
   Sorts integers in array
1  i ← 0
2  LOOP (i < size - 1)
3     smallest ← i
4     j ← i + 1
5     LOOP (j < size)
6        IF (list[j] < list[smallest])
7           smallest ← j
8        END IF
9        j ← j + 1
10    END LOOP
11    Swap(list[smallest], list[i])
12    i ← i + 1
13 END LOOP
END Sort
```

Sorting Algorithms

- Bubble Sort

```
ALGORITHM Sort(REF list[] <INTEGER>, size <INTEGER>)
   Sorts integers in array
1  i ← 1
2  LOOP (i < size)
3     j ← 0
4     LOOP (j < size - i)
5        IF (list[j] > list[j + 1])
6           Swap(list[j], list[j + 1])
7        END IF
8        j ← j + 1
9     END LOOP
10    i ← i + 1
11 END LOOP
END Sort
```

Questions